

Capítulo 3 — Docker: la receta reproducible

Unidad de estudio · Edición ampliada con fundamentos teóricos

Capítulo 3 — Docker: la receta reproducible	1
3.1. Alcance de la clase	2
3.2. Por qué existen los contenedores: origen y diseño	2
3.3. Qué es un contenedor: el modelo formal	4
3.3.1. Espacios de nombres y grupos de control	4
3.3.2. Imagen, capa y contenedor	4
3.3.3. Contenedor frente a máquina virtual	5
3.4. Anatomía de la construcción	6
3.4.1. El contexto de construcción	6
3.4.2. Qué instrucción crea una capa	6
3.4.3. La regla de invalidación del caché	7
3.5. El archivo de construcción del backend	7
3.5.1. Imagen base	7
3.5.2. Directorio de trabajo	8
3.5.3. El orden de copiado y el caché de capas	8
3.5.4. Declaración del puerto	8
3.5.5. El comando de arranque	9
3.5.6. Las dos formas de escribir un comando	9
3.6. El archivo <code>.dockerignore</code>	10
3.7. El archivo de construcción del frontend	11
3.7.1. El permiso de ejecución	11
3.7.2. Configuración en tiempo de arranque	11
3.8. Construcción y ejecución local	12
3.8.1. Backend	12
3.8.2. Frontend	13
3.8.3. Comandos de inspección	14
3.9. Buenas prácticas y evolución del ecosistema	14
3.9.1. El estándar OCI: Docker no es el único	14
3.9.2. Construcciones multietapa	14
3.9.3. No correr como root	15
3.9.4. Etiquetas: por qué <code>latest</code> es una trampa	15
3.10. Verificación	15
3.11. Errores frecuentes	16
3.12. Actividades	16
3.13. Síntesis	17

3.14. Referencias y lecturas complementarias 18

3.1. Alcance de la clase

Los alumnos llegan a esta instancia habiendo visto Docker de manera introductoria. El capítulo no repite esa introducción: se concentra en **leer y escribir el archivo de construcción**, porque es lo único que Easypanel va a ejecutar en el servidor, y porque sin entenderlo un fallo de construcción resulta indescifrable.

Ahora bien, "escribir el archivo de construcción" no se aprende memorizando instrucciones. Cada línea de un `Dockerfile` es una decisión que se justifica en un modelo: qué es una capa y por qué el orden importa, qué se envía al motor de construcción y qué no, por qué un proceso dentro de un contenedor ve un sistema de archivos que no es el del servidor. Este capítulo desarrolla ese modelo primero —incluida la historia de por qué los contenedores existen y qué problema vinieron a resolver— y recién después recorre los dos archivos línea por línea. La consecuencia es que, ante un fallo que no está en ninguna tabla de errores frecuentes, se pueda razonar en lugar de probar.

Al finalizar la clase, cada alumno debe tener los dos contenedores del proyecto construidos y funcionando en su propio equipo, comunicándose entre sí.

Contenidos

1. Origen y objetivos de diseño de la contenerización.
2. El modelo formal: espacios de nombres, grupos de control, imagen, capa y contenedor.
3. Contenedor frente a máquina virtual.
4. Anatomía de la construcción: contexto, instrucciones y sistema de caché.
5. El `Dockerfile` del backend, instrucción por instrucción.
6. El archivo `.dockerignore` y las filtraciones de secretos.
7. El `Dockerfile` del frontend y la generación de configuración en tiempo de arranque.
8. Construcción y ejecución local de ambos servicios.
9. Buenas prácticas y evolución del ecosistema: OCI, multietapa, usuario no privilegiado.
10. Diagnóstico de fallos de construcción.

3.2. Por qué existen los contenedores: origen y diseño

El problema que los contenedores vinieron a resolver tiene una formulación célebre y humillante: **"en mi máquina anda"**. La frase describe una situación real y cotidiana en la que un programa funciona en el equipo de quien lo escribió y falla en cualquier otro, no por un error del programa sino porque el entorno difiere: otra versión del intérprete, otra biblioteca del sistema, otra variable de entorno, otro paquete que estaba instalado desde hacía meses y que nadie recordaba haber instalado. El programa no es autosuficiente; depende de un contexto que nadie escribió en ninguna parte.

La primera respuesta industrial a ese problema fue la del capítulo anterior: **empaquetar la máquina entera**. Si el entorno es el problema, se distribuye el entorno completo como una imagen de máquina virtual. La solución funciona y es la base de la computación en la nube, pero paga un precio alto: cada máquina virtual lleva su propio sistema operativo completo, con su núcleo, sus servicios de arranque y sus gigabytes de disco, y tarda un minuto en encender. Para distribuir una aplicación de veinte megabytes, se movilizan dos gigabytes de sistema operativo.

La pregunta obvia es si hace falta virtualizar el hardware para aislar un proceso, y la respuesta — construida a lo largo de tres décadas en el mundo Unix— es que no. La pieza más antigua es de 1979: la llamada al sistema `chroot`, incorporada a la séptima edición de Unix, que permite cambiarle a un proceso cuál es su directorio raíz. A partir de ese momento, para ese proceso, / es otra cosa: no puede nombrar archivos fuera de ese subárbol simplemente porque no tiene forma de expresarlos. Es aislamiento del sistema de archivos, y solo de eso.

Las décadas siguientes agregaron las piezas faltantes. FreeBSD introdujo las *jails* en 2000, extendiendo la idea al espacio de procesos y a la red. Solaris hizo lo propio con las *zones* en 2004. En el mundo Linux, Google desarrolló desde 2006 el mecanismo de **grupos de control** (*cgroups*) para limitar el consumo de recursos de conjuntos de procesos, y lo incorporó al núcleo en 2008; en paralelo se fueron sumando los **espacios de nombres** (*namespaces*), que generalizan la idea de `chroot` a todo lo demás. Para 2013 el núcleo de Linux ya tenía todos los ingredientes, y estaban documentados, y eran gratis, y casi nadie los usaba: montarlos a mano requería un conocimiento profundo del núcleo.

Docker apareció ese año y su aporte **no fue tecnológico sino de empaquetado y de experiencia de uso**. Es importante decirlo con precisión, porque explica por qué se impuso tan rápido. Docker tomó mecanismos que ya existían y les puso encima tres decisiones de diseño.

Primera decisión: la receta es un archivo de texto, declarativo y versionable. El `Dockerfile` describe cómo se construye el entorno, y vive en el repositorio junto al código. El entorno deja de ser conocimiento tácito de una persona y pasa a ser un artefacto revisable, comparable entre versiones y ejecutable por una máquina. Es la misma idea que subyace a todo el movimiento de infraestructura como código.

Segunda decisión: el resultado es una imagen en capas, direccionada por contenido. Cada paso de la receta produce una capa identificada por un resumen criptográfico de su contenido. Dos imágenes que comparten los primeros pasos comparten físicamente esas capas: se descargan y se almacenan una sola vez. De ahí sale el sistema de caché de la sección 3.4.3 y de ahí sale que distribuir la centésima variante de una aplicación cueste megabytes y no gigabytes.

Tercera decisión: se reutiliza el núcleo del anfitrión. Un contenedor no arranca un sistema operativo: arranca un proceso. Por eso enciende en milisegundos y pesa lo que pesa la aplicación más sus dependencias. El precio de esa decisión es la contracara exacta de su ventaja, y conviene tenerlo presente: **el aislamiento de un contenedor es más débil que el de una máquina virtual**, porque todos los contenedores comparten el mismo núcleo, y una falla en ese núcleo los alcanza a todos.

PARA ENTENDER

La razón por la que existe todo esto se resume en una frase: **la receta es la misma en tu máquina y en el servidor**. Cuando Easypanel construya la imagen en el VPS, va a ejecutar exactamente el mismo archivo que vos corriste en tu notebook. "En mi máquina andaba" deja de ser una explicación válida, porque la máquina pasó a ser parte del código.

3.3. Qué es un contenedor: el modelo formal

3.3.1. Espacios de nombres y grupos de control

Un contenedor no es una entidad del núcleo de Linux. No existe una estructura de datos llamada "contenedor" ni una llamada al sistema que lo cree. **Un contenedor es un proceso común y corriente al que se le aplicaron dos familias de mecanismos**, y entender eso disuelve buena parte de la magia aparente.

Los **espacios de nombres** responden a la pregunta *¿qué ve este proceso?*. Cada espacio de nombres aísla un recurso global del sistema y le presenta al proceso su propia versión.

Espacio de nombres	Qué aísla	Efecto observable
<code>mnt</code>	El árbol de sistemas de archivos	El contenedor tiene su propio <code>/</code>
<code>pid</code>	Los identificadores de proceso	El proceso principal es el número 1
<code>net</code>	Interfaces, rutas y reglas de red	El contenedor tiene su propia IP (sección 2.8.1)
<code>uts</code>	Nombre de anfitrión	<code>hostname</code> devuelve el identificador del contenedor
<code>ipc</code>	Comunicación entre procesos	No ve las colas ni la memoria compartida del anfitrión
<code>user</code>	Identificadores de usuario y grupo	Se puede ser root adentro sin serlo afuera

Los **grupos de control** responden a otra pregunta: *¿cuánto puede consumir?*. Limitan y contabilizan el uso de procesador, memoria, entrada/salida de disco y ancho de banda de red de un conjunto de procesos. Sin ellos, un contenedor con una fuga de memoria se llevaría puesto el servidor entero.

Conviene notar cómo las dos filas destacadas de la tabla explican cosas ya vistas o por venir. El espacio de nombres de red es literalmente la razón de que un contenedor tenga dirección IP propia y de que su tráfico pase por `FORWARD` y no por `INPUT` (sección 2.8.1): para el núcleo es otra máquina. Y el espacio de nombres de procesos es la razón de que el proceso arrancado por `CMD` sea el número 1 del contenedor, con las consecuencias sobre el manejo de señales que se discuten en la sección 3.5.6.

3.3.2. Imagen, capa y contenedor

Tres conceptos, ahora con la precisión que el modelo permite.

Concepto	Definición	Analogía
Imagen	Un sistema de archivos empaquetado e inmutable	La receta y los ingredientes ya comprados
Contenedor	Una ejecución de una imagen	El plato efectivamente cocinado
Capa	Cada instrucción del <code>Dockerfile</code> produce una	Cada paso de la receta, guardado por separado

Una **capa** no es una copia completa del sistema de archivos: es la **diferencia** respecto de la capa anterior —qué archivos se agregaron, cuáles se modificaron y cuáles se marcaron como borrados— empaquetada y identificada por el resumen criptográfico SHA-256 de su contenido. Esa identificación por contenido es la que permite compartir capas entre imágenes: si dos imágenes distintas parten de `python:3.12-slim`, esa capa se guarda una vez y se referencia dos.

La imagen final se presenta al proceso como un único sistema de archivos coherente mediante un **sistema de archivos de unión** (en Linux moderno, OverlayFS): las capas se apilan y lo que se ve es el resultado de superponerlas, con las de arriba tapando a las de abajo. Sobre todas ellas, al ejecutar un contenedor, se agrega **una capa de escritura efímera**. Todo lo que el contenedor modifica se escribe ahí, con una técnica llamada *copia al escribir*: el archivo original de la capa inferior permanece intacto, y se copia hacia arriba solo cuando alguien lo toca.

De ese diseño se desprenden directamente dos hechos que en la práctica sorprenden:

- **Las imágenes no cambian nunca.** De una misma imagen pueden ejecutarse muchos contenedores simultáneos e independientes, y ninguno afecta a los demás ni a la imagen.
- **Todo lo que un contenedor escribe se pierde al eliminarlo**, porque vivía en la capa efímera. Ese es exactamente el problema que los volúmenes resuelven, y es el contenido de la sección 5.6.3.

⚠ OJO ACÁ

De acá sale una consecuencia de seguridad que se retoma en la sección 3.6 y conviene anticipar: **una capa no se puede editar, solo tapar**. Si en un paso copiaste un archivo con credenciales y en un paso posterior lo borraste, la capa que lo contiene **sigue estando en la imagen** y cualquiera con la imagen puede extraerlo. Borrar un secreto en una capa siguiente no lo elimina: lo esconde.

3.3.3. Contenedor frente a máquina virtual

La comparación con el capítulo anterior ordena las dos tecnologías, que resuelven problemas emparentados con compromisos opuestos.

	Máquina virtual	Contenedor
Qué se virtualiza	El hardware	El espacio de nombres del sistema operativo
Núcleo	Uno propio por máquina	Compartido con el anfitrión
Tiempo de arranque	Decenas de segundos	Milisegundos
Tamaño típico	Gigabytes	Megabytes
Fuerza del aislamiento	Fuerte (lo impone el procesador)	Menor (lo impone el núcleo)
Puede correr otro sistema operativo	Sí	No: comparte el núcleo

La última fila explica una pregunta recurrente: no existen "contenedores Linux sobre Windows" en sentido estricto. Docker Desktop en Windows levanta una máquina virtual Linux —usando la tecnología del capítulo 2— y corre los contenedores adentro. Las dos tecnologías no compiten: se apilan. El VPS del práctico es una máquina virtual, y dentro de esa máquina virtual corren contenedores.

 DATO

Que el aislamiento del contenedor sea "menor" no significa que sea débil: significa que su superficie de ataque es el núcleo de Linux, que es enorme, en lugar de la interfaz del procesador, que es diminuta. En la práctica, para un despliegue como el de este módulo es más que suficiente. Importa saberlo cuando alguien propone correr en el mismo servidor código de terceros no confiable: ahí la respuesta correcta ya no es un contenedor.

3.4. Anatomía de la construcción

3.4.1. El contexto de construcción

Al ejecutar `docker build .`, ese punto final no significa "construí acá": significa enviá el contenido de este directorio al motor de construcción. Ese conjunto de archivos se llama contexto de construcción, y la operación es una transferencia real, aunque el motor esté en la misma máquina.

Dos consecuencias que explican comportamientos frecuentes. La primera: la construcción **no puede acceder a ningún archivo fuera del contexto**. Un `COPY ../otro-proyecto/config.json` falla siempre, y no por una restricción arbitraria sino porque ese archivo nunca se envió. La segunda: si el directorio contiene un entorno virtual de Python de 400 MB o una carpeta `node_modules`, todo eso se transfiere en cada construcción, aunque ninguna instrucción lo use. De ahí la importancia del `.dockerignore` de la sección 3.6, que la mayoría de los tutoriales presenta como una optimización menor y que en realidad gobierna qué tan rápido y qué tan seguro es construir.

3.4.2. Qué instrucción crea una capa

No todas las instrucciones del `Dockerfile` son iguales. Unas modifican el sistema de archivos y por lo tanto producen una capa; otras solo escriben metadatos en la imagen.

Instrucción	¿Crea capa?	Qué hace
FROM	La hereda	Establece la imagen base
COPY / ADD	Sí	Agrega archivos al sistema de archivos
RUN	Sí	Ejecuta un comando y guarda el resultado
WORKDIR	No (metadato)	Fija el directorio de trabajo
ENV	No (metadato)	Define una variable de entorno
EXPOSE	No (metadato)	Documenta un puerto
CMD / ENTRYPOINT	No (metadato)	Declara el proceso de arranque
USER	No (metadato)	Fija el usuario de ejecución

Distinguir las importa por dos motivos. Uno práctico: solo las instrucciones que crean capa pesan, y por eso se recomienda encadenar varios comandos en un único `RUN` con `&&` en lugar de escribir tres `RUN` seguidos. Uno conceptual: las instrucciones de metadato no ejecutan nada —`EXPOSE` no abre un puerto, `ENV` no configura el sistema operativo del anfitrión—, y esperar que lo hagan es una fuente clásica de confusión.

3.4.3. La regla de invalidación del caché

Docker almacena el resultado de cada instrucción y lo reutiliza si nada cambió. La regla exacta, que conviene poder enunciar, tiene dos partes:

1. Para la mayoría de las instrucciones, la clave del caché es **el texto literal de la instrucción** más la identidad de la capa anterior. Si el texto es idéntico y la capa previa es la misma, se reutiliza sin ejecutar nada.
2. Para COPY y ADD, la clave incluye además un resumen del contenido de los archivos copiados. Cambia un byte de un archivo copiado, cambia la clave.

Y por encima de ambas, la consecuencia decisiva: **cuando una capa se invalida, todas las siguientes se invalidan también**, aunque su texto no haya cambiado. El caché es una cadena, no un conjunto: no se puede reutilizar el eslabón número cinco si el cuatro se rompió.

Esa propiedad en cascada es la que convierte el orden de las instrucciones en una decisión de ingeniería y no en una cuestión de gusto, y es lo que se aplica en la sección 3.5.3.

PARA ENTENDER

Fijate en la forma de la regla, porque el patrón se repite: **lo que cambia poco va arriba, lo que cambia mucho va abajo**. Las dependencias cambian una vez por mes; el código cambia veinte veces por día. Ponerlos en ese orden no es una micro-optimización: es la diferencia entre que un despliegue tarde diez segundos o cuatro minutos, multiplicado por cada vez que alguien hace un push.

3.5. El archivo de construcción del backend

El archivo completo se encuentra en `backend/Dockerfile`. Se analiza instrucción por instrucción.

3.5.1. Imagen base

```
FROM python:3.12-slim
```

Toda imagen parte de otra. `python:3.12-slim` es una distribución mínima de Debian con Python 3.12 ya instalado.

Variante	Tamaño aproximado	Cuándo se usa
<code>python:3.12</code>	~1 GB	Cuando hacen falta herramientas de compilación
<code>python:3.12-slim</code>	~150 MB	La que se usa acá. Suficiente para dependencias puras
<code>python:3.12-alpine</code>	~50 MB	Más chica, pero incompatible con muchos paquetes binarios

La elección de la imagen base es la decisión de mayor impacto de todo el archivo, porque fija el tamaño de partida, el conjunto de bibliotecas del sistema disponibles y —esto se subestima siempre— **la superficie de vulnerabilidades heredada**. Una imagen base con doscientos paquetes instalados trae las vulnerabilidades de esos doscientos paquetes, se usen o no. La regla es la economía del mecanismo de la sección 2.12.1 aplicada acá: la imagen más chica que alcance.

DATO

alpine usa una biblioteca de C distinta (musl en lugar de glibc). Eso hace que los paquetes de Python distribuidos como binarios precompilados no sirvan, y pip tenga que compilarlos desde el código fuente. El resultado es una imagen más chica a costa de construcciones mucho más lentas, y a veces directamente fallidas. Para este proyecto, slim es la elección correcta.

3.5.2. Directorio de trabajo

```
WORKDIR /app
```

Fija el directorio donde se ejecutan las instrucciones siguientes, y lo crea si no existe. Evita tener que escribir rutas absolutas en cada línea. Es una instrucción de metadato: no crea capa y no ejecuta nada.

3.5.3. El orden de copiado y el caché de capas

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .
```

Esta es la parte del archivo donde más conviene detenerse, porque la decisión no es evidente: podría copiarse todo junto en una sola instrucción y el resultado funcional sería idéntico. Lo que cambia no es qué imagen se produce sino **cuánto cuesta producirla la próxima vez**, y es la aplicación literal de la regla de la sección 3.4.3.

Con el orden del archivo:

Qué se modificó	pip install se ejecuta	Duración
Solo main.py	No, sale del caché	Segundos
requirements.txt	Sí	Minutos

Si ambos archivos se copiaran en la misma instrucción, cualquier cambio en main.py —incluso una coma— cambiaría el resumen del contenido copiado, invalidaría esa capa y, por la propiedad en cascada, obligaría a volver a descargar todas las dependencias.

EXPERIMENTO

Construí la imagen, cambiá un comentario en main.py, construí de nuevo y mirá el tiempo. Después invertí el orden de las instrucciones en el Dockerfile, repetí la operación y compará. La diferencia es de dos órdenes de magnitud, y sobre esa diferencia se apoya todo el flujo de despliegue: cada push que hagan va a reconstruir la imagen en el servidor.

La opción --no-cache-dir de pip es un asunto distinto y conviene no confundirla con el caché de Docker: impide que pip guarde dentro de la imagen una copia de los paquetes descargados. Como esos archivos no se van a volver a usar, solo agregarían peso a la capa. Son dos cachés diferentes, con nombres parecidos, que no tienen nada que ver entre sí.

3.5.4. Declaración del puerto

```
EXPOSE 8000
```

 **OJO ACÁ**

EXPOSE no abre nada. Es documentación: le avisa a quien lea el archivo, y a herramientas como Easypanel, en qué puerto escucha la aplicación. Si lo borrás, el contenedor funciona exactamente igual. Y si lo ponés mal, tampoco pasa nada — pero vas a confundir a la próxima persona que lo lea, que probablemente seas vos dentro de seis meses.

Que sea metadato no significa que sea inútil. Easypanel lo lee para proponer a qué puerto enrutar, y `docker run -P` (con P mayúscula) publica automáticamente todos los puertos declarados con EXPOSE en puertos libres del anfitrión. Es información que otras herramientas consumen; lo que no hace es actuar por sí misma sobre la red.

3.5.5. El comando de arranque

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

CMD define el proceso que se ejecuta al iniciar el contenedor. Cuando ese proceso termina, el contenedor se detiene: el ciclo de vida del contenedor es el ciclo de vida de ese proceso, y no hay nada más adentro que lo mantenga en pie.

El parámetro `--host 0.0.0.0` es el error más frecuente al llevar una API a un contenedor, y retoma la distinción de interfaces de la sección 2.6.1. Con el modelo de la sección 3.3.1 a mano, el motivo ahora es transparente: el contenedor tiene su propio espacio de nombres de red y su propia interfaz de bucle local. Escuchar en `127.0.0.1` significa escuchar en el bucle local del contenedor, que es un lugar al que nadie más puede llegar, ni siquiera el anfitrión.

Valor	Significado dentro del contenedor	Consecuencia
<code>127.0.0.1</code>	Solo este contenedor	Nadie de afuera puede conectarse
<code>0.0.0.0</code>	Todas las interfaces	Accesible desde el proxy y desde otros servicios

⚠ OJO ACÁ

Este fallo es especialmente cruel porque **no se parece a un fallo**. El contenedor arranca, el log dice `Uvicorn running on http://127.0.0.1:8000`, no hay ningún error en ningún lado, y el servicio no contesta nunca. Ni a Easypanel, ni a Traefik, ni a vos. En la Clase 4 se manifiesta como un 502 Bad Gateway y manda a buscar el problema a cualquier lado menos al correcto.

3.5.6. Las dos formas de escribir un comando

La instrucción anterior está escrita como una lista de cadenas entre corchetes. Esa notación se llama **forma de ejecución** (*exec form*), y la alternativa —escribir el comando como texto suelto— se llama **forma de intérprete** (*shell form*). La diferencia parece cosmética y no lo es.

Forma	Cómo se escribe	Qué ocurre
Ejecución	<code>CMD ["uvicorn", "main:app"]</code>	El programa se ejecuta directamente y es el proceso número 1
Intérprete	<code>CMD uvicorn main:app</code>	Se ejecuta <code>/bin/sh -c "uvicorn main:app"</code> ; el intérprete es el proceso 1

La consecuencia está en el apagado. Cuando se detiene un contenedor, Docker envía la señal SIGTERM al proceso número 1 para que cierre ordenadamente, espera unos segundos y, si no

terminó, lo mata con SIGKILL. En la forma de intérprete, el proceso número 1 es el intérprete de comandos, que no reenvía la señal al programa que lanzó: la aplicación nunca se entera de que la están apagando, no cierra sus conexiones ni termina lo que estaba haciendo, y muere de golpe al vencer el plazo. El síntoma visible es que cada `docker stop` tarda diez segundos.

La regla es simple: **usar siempre la forma de ejecución**, con corchetes y comillas dobles, salvo que se necesite expresamente una funcionalidad del intérprete —expansión de variables, tuberías, redirecciones—, en cuyo caso lo correcto es escribir un script y llamarlo desde la forma de ejecución.

DATO

Este detalle explica un fenómeno que se ve todo el tiempo y que se suele atribuir a "que Docker es lento": contenedores que tardan exactamente diez segundos en detenerse. No es lentitud, es el plazo de gracia venciendo. Un contenedor bien escrito se detiene en menos de un segundo.

3.6. El archivo `.dockerignore`

Define qué **no** se envía al contexto de construcción de la sección 3.4.1. Cumple tres funciones, y la tercera es de otra categoría que las dos primeras.

Función	Ejemplo
Reducir el tamaño	Excluir <code>__pycache__/, .venv/</code>
Preservar el caché	Que un archivo temporal no invalide la capa de COPY
Evitar filtraciones	Que un <code>.env</code> no termine dentro de la imagen

OJO ACÁ

La tercera función es la seria, y ahora que está a mano el modelo de capas de la sección 3.3.2 se entiende por qué. Si tenés un archivo con credenciales en la carpeta y hacés `COPY . .`, ese archivo queda adentro de la imagen para siempre, aunque después lo borres en una instrucción posterior: la capa que lo contiene sigue ahí, es extraíble con herramientas comunes, y viaja con la imagen a donde sea que la publiques.

Y si eso pasó, borrar el archivo no alcanza: **la credencial hay que rotarla**. Un secreto que salió de tu máquina es un secreto quemado.

Un `.dockerignore` mínimo y razonable para este proyecto excluye el control de versiones, los entornos virtuales, los cachés del intérprete, los archivos de configuración local y todo lo que solo sirva para desarrollo:

```
.git/
.github/
__pycache__/
.venv/
*.pyc
.env
requirements-dev.txt
```

3.7. El archivo de construcción del frontend

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html
COPY docker-entripoint.sh /docker-entripoint.d/40-generar-config.sh
RUN chmod +x /docker-entripoint.d/40-generar-config.sh

EXPOSE 80
```

El contraste con el backend es deliberado y conviene detenerse en él.

	Backend	Frontend
¿Ejecuta código propio en el servidor?	Sí, Python	No
Qué hace la imagen	Instala dependencias y corre un proceso	Entrega archivos
Dónde se ejecuta el código de la aplicación	En el servidor	En el navegador del visitante
Cantidad de instrucciones	7	5

PARA ENTENDER

Fijate que en el frontend **no hay** CMD. ¿Por qué? Porque la imagen oficial de nginx ya trae el suyo, y se hereda. Nosotros no necesitamos cambiar cómo arranca nginx: solo necesitamos poner archivos donde nginx los busca.

Y fijate en algo más grande: el frontend no ejecuta **nada** en el servidor. El JavaScript de la calculadora corre en la máquina del visitante. Esa observación, que ahora parece un detalle, es la que en la Clase 5 explica por qué el frontend no puede usar la red interna aunque quisiera.

3.7.1. El permiso de ejecución

```
RUN chmod +x /docker-entripoint.d/40-generar-config.sh
```

La imagen oficial de nginx ejecuta automáticamente, antes de iniciar el servidor, todos los archivos que encuentre en `/docker-entripoint.d/`, en orden alfabético. El prefijo numérico `40-` sirve para ordenarlos. Es una convención de esa imagen en particular, no una regla de Docker: la imagen de PostgreSQL tiene su equivalente en `/docker-entripoint-initdb.d/`, y conviene revisar la documentación de cada imagen base antes de suponer.

Pero solo ejecuta los archivos que tienen permiso de ejecución.

OJO ACÁ

Esa línea de `chmod` existe porque Windows no guarda el permiso de ejecución de los archivos. Si desarrollás en Windows, el script se copia sin ese permiso, nginx lo ignora sin decir una palabra, `config.js` no se genera, y el frontend queda apuntando a la nada. El contenedor arranca perfecto y la aplicación no anda. Sin el `chmod` explícito, este proyecto funciona en Linux y falla en Windows.

3.7.2. Configuración en tiempo de arranque

El script `docker-entripoint.sh` escribe `config.js` leyendo la variable de entorno `API_URL`:

```
cat > /usr/share/nginx/html/config.js <<EOF
window.CONFIG = {
  API_URL: "${API_URL}",
};
EOF
```

La decisión de generar el archivo **al arrancar** y no al construir tiene un fundamento concreto: este frontend no tiene etapa de compilación. Es HTML que el navegador lee tal cual, sin ningún momento intermedio en el que se le pudiera inyectar la dirección de producción.

Momento	Qué implicaría	Problema
Al construir	La URL queda dentro de la imagen	Una imagen distinta por cada entorno
Al arrancar	La URL viene de una variable	Una sola imagen para todos los entornos

Esta decisión tiene nombre propio en la literatura. La metodología conocida como *doce factores*, formulada en 2011 a partir de la experiencia operativa de una plataforma de despliegue, enuncia dos principios que son exactamente lo que la tabla anterior describe: la configuración se almacena **en el entorno**, estrictamente separada del código; y las etapas de **construcción, publicación y ejecución** son tres momentos distintos que no deben mezclarse. Una imagen que lleva adentro la dirección de producción viola las dos: mezcla configuración con código, y mezcla construcción con ejecución.

PARA ENTENDER

El principio general, que aplica muchísimo más allá de este proyecto: **la configuración viene del entorno, no del código**. El mismo `index.html`, el mismo `Dockerfile` y la misma imagen funcionan en tu notebook y en producción. Lo único que cambia es una variable de entorno. Si tenés que reconstruir para cambiar de entorno, algo está mal diseñado.

Nótese además que el script avisa por el log si la variable no está definida, en lugar de generar un archivo silenciosamente incorrecto. Un contenedor que arranca "bien" pero mal configurado es mucho más difícil de diagnosticar que uno que se queja.

3.8. Construcción y ejecución local

Ambos repositorios se clonan por separado, dado que el proyecto está organizado en dos repositorios independientes.

```
git clone https://github.com/MatyAlts/calculadora-backend.git
git clone https://github.com/MatyAlts/calculadora-frontend.git
```

3.8.1. Backend

Situado en el directorio del backend:

```
docker build -t calculadora-api .
docker run -p 8000:8000 -e ORIGENES_PERMITIDOS=http://localhost:8080 calculadora-api
```

Parámetro	Significado
-t calculadora-api	Nombre de la imagen resultante
.	Contexto de construcción: el directorio actual (sección 3.4.1)
-p 8000:8000	Publica el puerto: puerto_del_host:puerto_del_contenedor

Parámetro	Significado
-e VARIABLE=valor	Define una variable de entorno

Verificación:

```
curl http://localhost:8000/api/salud
```

3.8.2. Frontend

Situado en el directorio del frontend:

```
docker build -t calculadora-web .
docker run -p 8080:80 -e API_URL=http://localhost:8000 calculadora-web
```

Obsérvese que el mapeo es 8080:80: nginx escucha en el puerto 80 dentro del contenedor, y se publica en el 8080 del equipo para no requerir privilegios de administrador. Es la restricción de los puertos bien conocidos de la sección 2.6.2, y es un buen ejemplo de que el puerto de adentro y el de afuera no tienen por qué coincidir.

En el registro de arranque debe aparecer:

```
config.js generado con API_URL=http://localhost:8000
```

Y la aplicación completa queda disponible en <http://localhost:8080>.

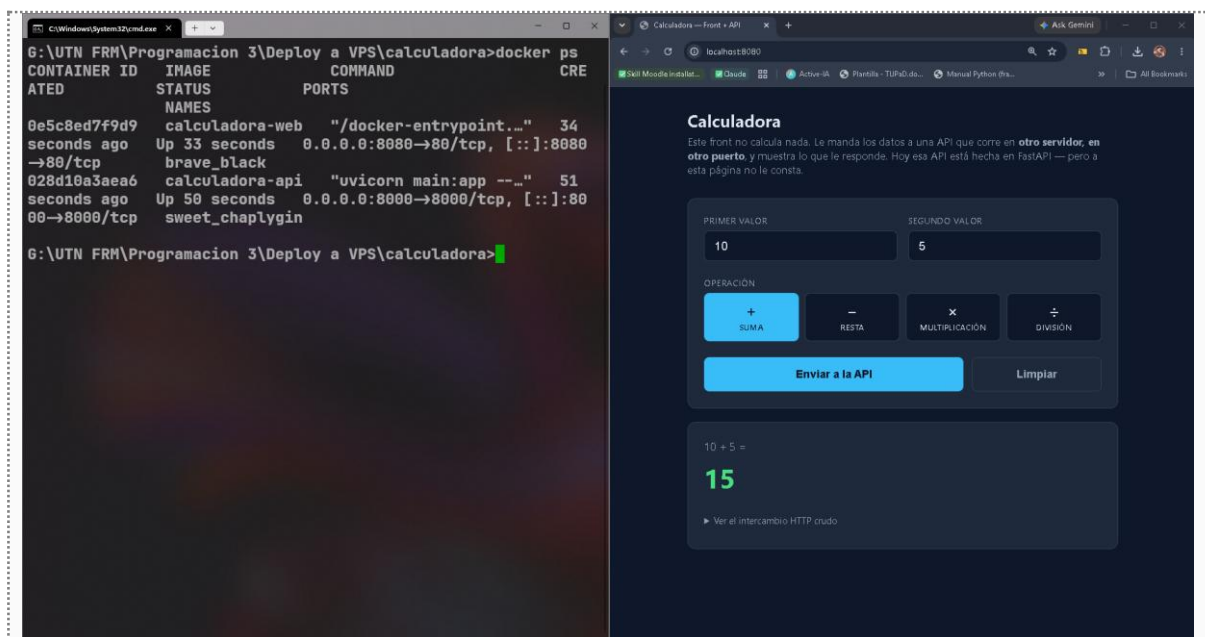


Figura 3.1. Los dos servicios del proyecto ejecutándose en el equipo del alumno.

⚠ OJO ACÁ

El valor de `ORIGENES_PERMITIDOS` del backend tiene que ser exactamente `http://localhost:8080`, que es el origen desde el que el navegador va a hacer el pedido. Si ponés 8000, o 127.0.0.1, o le agregás una barra al final, el navegador bloquea la petición. Es el mismo CORS del README.md del proyecto, ahora con puertos distintos. Si no te acordás por qué, releelo: es lectura previa de la Clase 4.

3.8.3. Comandos de inspección

Objetivo	Comando
Ver contenedores en ejecución	<code>docker ps</code>
Ver el registro de un contenedor	<code>docker logs <id></code>
Seguir el registro en vivo	<code>docker logs -f <id></code>
Abrir una consola dentro del contenedor	<code>docker exec -it <id> sh</code>
Detener un contenedor	<code>docker stop <id></code>
Ver las capas y el tamaño de una imagen	<code>docker history calculadora-api</code>
Ver la configuración completa de una imagen	<code>docker inspect calculadora-api</code>

`docker history` merece un comentario porque es la ventana directa al modelo de la sección 3.3.2: muestra cada capa, la instrucción que la creó y cuánto pesa. Es la herramienta con la que se descubre que una imagen de 900 MB tiene 700 MB en una sola capa de `RUN apt-get install`, y también con la que se comprueba que un archivo que se creía borrado sigue ahí.

EXPERIMENTO

Entrá al contenedor del frontend con `docker exec -it <id> sh` y mirá el archivo `config.js` generado:

```
cat /usr/share/nginx/html/config.js
```

Comprabá con tus propios ojos que ese archivo **no existía en el repositorio con ese contenido**: lo escribió el script al arrancar, con el valor de la variable de entorno. Es la forma más directa de entender qué significa "configuración en tiempo de arranque".

Aprovechá que estás adentro y corré también `hostname`, `ls /` y `ps aux`. Vas a ver un nombre de máquina que no es el tuyo, un sistema de archivos que no es el tuyo y una lista de procesos donde `nginx` es el número 1. Son los espacios de nombres de la sección 3.3.1, funcionando delante de tus ojos.

3.9. Buenas prácticas y evolución del ecosistema

3.9.1. El estándar OCI: Docker no es el único

En 2015, con la contenerización ya consolidada, la industria formalizó los formatos para que no quedaran atados a un producto. La **Open Container Initiative** publica tres especificaciones: la de **imagen** (cómo se estructura una imagen en capas y sus metadatos), la de **ejecución** (cómo se arranca un contenedor a partir de un directorio y una configuración) y la de **distribución** (cómo funciona un registro de imágenes).

La consecuencia práctica es que "imagen de Docker" es hoy un nombre histórico: lo que se construye es una **imagen OCI**, y puede ejecutarse con Docker, con Podman, con containerd o con cualquier otro motor conforme. El `Dockerfile` que se escribe en esta clase no ata el proyecto a ninguna empresa, y esa es exactamente la garantía que la estandarización buscaba.

3.9.2. Construcciones multietapa

Cuando una aplicación necesita herramientas para construirse que no necesita para ejecutarse —un compilador, un empaquetador de JavaScript, las cabeceras de desarrollo de una biblioteca—, incluirlas en la imagen final es cargar peso y superficie de ataque sin ninguna contraprestación. La

solución estándar es la **construcción multietapa**: se declaran varias instrucciones FROM en el mismo archivo, se hace el trabajo pesado en la primera y se copia a la última solo el resultado.

```
FROM node:20 AS construccion
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=construccion /app/dist /usr/share/nginx/html
```

La imagen final contiene nginx y los archivos generados: ni Node, ni node_modules, ni el código fuente. Este proyecto no la necesita —el frontend es HTML sin compilación— pero es el patrón que va a aparecer en el primer proyecto real con React, Vue o Angular, y conviene reconocerlo.

3.9.3. No correr como root

Por omisión, el proceso de un contenedor corre como root dentro del contenedor. Gracias al espacio de nombres de usuario eso no equivale a ser root en el anfitrión, pero sigue siendo más privilegio del necesario: si un atacante logra ejecutar código dentro del contenedor, prefiere encontrarse con un usuario sin permisos que con el administrador. La instrucción USER lo corrige:

```
RUN useradd --create-home aplicacion
USER aplicacion
```

Es mínimo privilegio (sección 2.12.1) aplicado adentro de la imagen. Nótese que debe ir **después** de las instrucciones que necesitan privilegios —instalar paquetes, cambiar permisos— y antes del CMD.

3.9.4. Etiquetas: por qué latest es una trampa

Una imagen se referencia por nombre y **etiqueta**: python:3.12-slim. Si no se escribe etiqueta, Docker asume latest, y ahí está el problema: latest no significa "la más nueva". Es simplemente el nombre de etiqueta que se usa por omisión, y apunta a lo que el publicador haya decidido, que puede cambiar de un día para el otro sin aviso.

La consecuencia es que un FROM python:latest puede construir hoy con Python 3.12 y dentro de tres meses con Python 3.14, rompiendo el proyecto sin que nadie haya tocado una línea. La práctica correcta es fijar la versión —como hace el archivo de este proyecto— y, en contextos que exigen reproducibilidad exacta, fijar directamente el resumen criptográfico de la imagen (python@sha256:...), que por construcción no puede cambiar nunca.

DATO

La misma lógica se aplica a requirements.txt. Este proyecto fija versiones exactas con ==, no rangos con >=. Es más trabajo de mantenimiento y es la única forma de que la construcción de hoy y la de dentro de seis meses produzcan lo mismo. Una construcción reproducible no es la que funciona: es la que vuelve a funcionar igual.

3.10. Verificación

#	Comprobación	Resultado esperado
1	docker build en ambos repositorios	Finaliza sin errores

#	Comprobación	Resultado esperado
2	Segunda construcción tras cambiar solo el código	Sensiblemente más rápida
3	<code>docker ps</code>	Dos contenedores en ejecución
4	<code>curl http://localhost:8000/api/salud</code>	<code>{"estado": "ok"}</code>
5	<code>http://localhost:8080</code> en el navegador	La calculadora carga
6	Una operación completa desde el navegador	Resultado correcto, sin errores en consola
7	<code>config.js</code> dentro del contenedor	Contiene el valor de <code>API_URL</code>
8	<code>docker stop</code> sobre el backend	Se detiene en menos de dos segundos

La comprobación 2 valida el modelo de caché de la sección 3.4.3; la 7, la configuración en tiempo de arranque de la sección 3.7.2; y la 8, la forma de ejecución del CMD según la sección 3.5.6. Cada una interroga una pieza distinta.

3.11. Errores frecuentes

Mensaje o síntoma	Causa	Resolución
failed to read dockerfile	Se ejecutó <code>docker build</code> fuera del directorio correcto	Situarse donde está el Dockerfile
COPY failed: file not found	El archivo no existe, está fuera del contexto o excluido por <code>.dockerignore</code>	Verificar nombre, ubicación y <code>.dockerignore</code>
port is already allocated	Otro proceso usa ese puerto del equipo	Cambiar el puerto del host o detener el otro proceso
El contenedor arranca y se detiene solo	El proceso de CMD terminó	<code>docker logs</code> para ver el motivo
La API no responde desde el navegador	<code>--host 127.0.0.1</code> en lugar de <code>0.0.0.0</code>	Corregir el CMD (sección 3.5.5)
<code>config.js</code> no se genera	Falta el permiso de ejecución del script	Verificar el <code>chmod +x</code>
La construcción tarda siempre lo mismo	El orden de las instrucciones invalida el caché	Copiar dependencias antes que el código
Cada <code>docker stop</code> tarda diez segundos	El CMD está en forma de intérprete	Pasarlo a forma de ejecución (sección 3.5.6)
El envío del contexto tarda minutos	Falta <code>.dockerignore</code> y se envía <code>.venv/</code> o <code>.git/</code>	Escribir el <code>.dockerignore</code>

⚠ OJO ACÁ

Cuando falla una construcción, **el error importante es el primero, no el último**. Docker sigue imprimiendo mensajes después de romperse, y el final de la salida suele ser ruido. Léete de arriba hacia abajo. Este consejo, que parece obvio, es el que más tiempo les va a ahorrar en la Clase 4.

3.12. Actividades

Actividad 1 — Medición del caché.

Registrar el tiempo de tres construcciones consecutivas: la inicial, una tras modificar `main.py` y una tras modificar `requirements.txt`. Explicar las diferencias a partir de la regla de invalidación de la sección 3.4.3, indicando en cada caso a partir de qué instrucción se rompió la cadena.

Actividad 2 — Ruptura deliberada.

Cambiar `--host 0.0.0.0` por `--host 127.0.0.1`, reconstruir y observar el comportamiento.

Documentar qué muestra el registro y qué muestra el navegador. Explicar el fenómeno en términos del espacio de nombres de red de la sección 3.3.1. Restaurar después.

Actividad 3 — Inversión del orden.

Reemplazar las tres instrucciones de copiado por un único `COPY` . . y medir el impacto sobre el tiempo de reconstrucción tras un cambio en el código.

Actividad 4 — Inspección de capas.

Ejecutar `docker history` sobre ambas imágenes y comparar cantidad de capas y tamaño total. Identificar cuál es la capa más pesada de cada una y justificar por qué. Contrastar el resultado con la tabla de instrucciones de la sección 3.4.2: ¿alguna instrucción de metadato aparece con tamaño?

Actividad 5 — Configuración por entorno.

Levantar el frontend con tres valores distintos de `API_URL` sin reconstruir la imagen, y verificar en cada caso el contenido de `config.js`. Relacionar el resultado con los principios de configuración y de separación entre construcción y ejecución de la sección 3.7.2.

Actividad 6 — Los espacios de nombres, en vivo.

Con el backend corriendo, ejecutar `docker exec -it <id> sh` y desde adentro correr `hostname`, `ip addr`, `ps aux` y `ls /`. Confeccionar una tabla que asocie cada salida observada con el espacio de nombres de la sección 3.3.1 que la produce. Comparar cada resultado con el del mismo comando ejecutado en el anfitrión.

Actividad 7 — Un secreto que no se borra. (demostrativa)

Crear un archivo `secreto.txt` con un texto reconocible, agregar a un `Dockerfile` de prueba una instrucción que lo copie y otra posterior que lo borre, y construir la imagen. Verificar con `docker history` que la capa sigue presente. Explicar por qué el archivo sigue siendo recuperable y qué debería hacerse si el secreto hubiera sido real.

3.13. Síntesis

1. Los contenedores resuelven el problema de la **reproducibilidad del entorno** sin el costo de virtualizar el hardware. El aporte de Docker no fue tecnológico: fue la receta declarativa, la imagen en capas y la reutilización del núcleo del anfitrión.
2. Un contenedor **no es una entidad del núcleo**: es un proceso con espacios de nombres —que definen qué ve— y grupos de control —que definen cuánto consume—.
3. Una imagen es **inmutable** y está compuesta de capas identificadas por su contenido. Una capa no se puede editar, solo tapar: **un secreto copiado queda en la imagen para siempre**.
4. El contexto de construcción se **envía entero** al motor. Lo que no se excluye, viaja.
5. Cada instrucción produce una capa o un metadato, y **cuando una capa se invalida, todas las siguientes también**. Por eso lo que cambia poco va arriba y lo que cambia mucho va abajo.
6. **EXPOSE no abre puertos**. Es documentación que otras herramientas consumen.
7. Dentro de un contenedor, **0.0.0.0 es obligatorio** para que el servicio sea alcanzable: el bucle local del contenedor no lleva a ninguna parte.
8. La **configuración viene del entorno**, no del código. Una sola imagen sirve para todos los entornos, y construcción y ejecución son etapas separadas.
9. Al diagnosticar una construcción fallida, **el primer error es la causa**; el resto es consecuencia.

3.14. Referencias y lecturas complementarias

Las especificaciones normativas del formato ya no pertenecen a Docker sino a la **Open Container Initiative**, y están publicadas en github.com/opencontainers: la Image Specification define la estructura de una imagen en capas y sus metadatos, la Runtime Specification define cómo se ejecuta un contenedor a partir de un sistema de archivos y una configuración, y la Distribution Specification normaliza el protocolo de los registros de imágenes. La documentación oficial de la instrucción Dockerfile y la guía de buenas prácticas de construcción, en docs.docker.com, son la referencia operativa cotidiana.

El artículo que presentó la tecnología al público general es D. Merkel, *Docker: Lightweight Linux Containers for Consistent Development and Deployment* (*Linux Journal*, n.º 239, 2014). Para los mecanismos del núcleo sobre los que todo esto se apoya —espacios de nombres, grupos de control y llamadas al sistema asociadas—, la referencia exhaustiva es M. Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), complementada por las páginas de manual `namespaces(7)` y `cgroups(7)`, que se leen bien y están en cualquier sistema Linux.

La metodología de los **doce factores**, formulada por A. Wiggins en 2011 y disponible en 12factor.net, es breve y vale la pena leerla completa: los factores III (Config) y V (Build, release, run) son exactamente el fundamento de la sección 3.7.2. Como manual de estudio, J. Nickoloff y S. Kuenzli, *Docker in Action* (2.ª edición, Manning, 2019), cubre con detalle el modelo de imágenes, los volúmenes y las redes que se retoman en la Clase 5.